Dynamic Scheduling without Real-Time Requirements

Moiz Zaheer Malik
B.Eng. Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
moiz-zaheer.malik@stud.hshl.de

***Abstract*—High-Level Synthesis (HLS) usually uses fixed scheduling to meet timing and performance goals. But when real-time deadlines aren't required, we can use dynamic scheduling to improve energy efficiency. Instead of assigning operations to specific clock cycles ahead of time, dynamic methods can decide the order and timing of tasks during runtime. This helps reduce power usage by cutting down on unnecessary activity and allowing parts of the hardware to turn off when they're not needed.**

This paper looks at a dynamic scheduling method called the Power Scheduler, proposed by Rettberg and Rammig[7]. Their approach uses ASAP and ALAP scheduling to analyze the Control/Data Flow Graph (CDFG), then builds a compatibility graph to group operations that can be powered down together. This way, operations with flexible timing (called mobility) can be rescheduled to save energy without breaking the program.

A case study on MPEG-2 color conversion using the MACT self-timed pipeline shows that this method can cut power use by about 10-15%, with only small performance losses[4]. Compared to regular static scheduling, this approach is a better fit for energy-conscious embedded system designs.

***Index Terms*—dynamic scheduling, high-level synthesis, power optimization, partitioning, CDFG, MACT architecture**

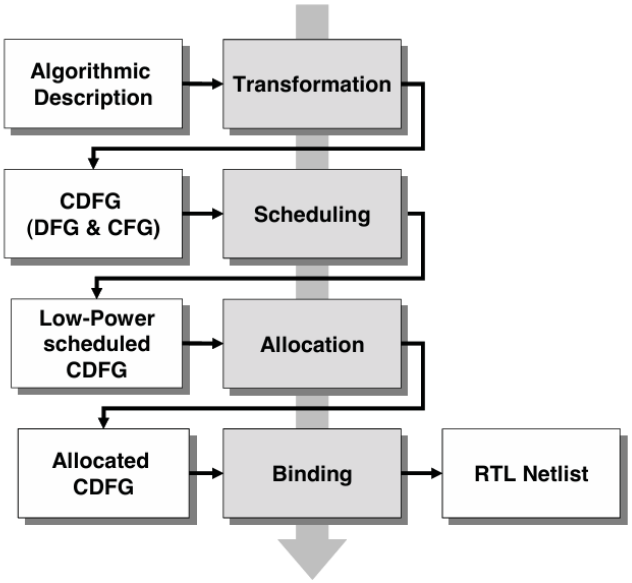

Fig. 1. Design phases of the High-Level Synthesis [7]

I. Introduction

In today's digital world, many systems and wearable devices like smart watches and smartphones, and other embedded electronics are expected tp perform more tasks while using really little energy and power. Especially in battery powered and portable devices power efficincy is just as important as speed or performance[1].

A. High Level Synthesis and the Role of Dynamic Scheduling

Dynamic schedulling is a really important technique in modern hardware design, mostly in the domain of High Level Synthesis (HLS)[7], where we convert a high level description like (C or SystemC code) into low-level Hardware Description Languages like (VHDL / Verligo).

The key advantage of of HLS is that it lets the designers to work at a high level of abstraction. insted of worring and focusing on gates, flipflops, and clock cycles, it helps them to describe the algorithms and the behaviour of system more naturally and makes it simple for them, just like how the write software. The HLS tool then translates that high level description into the register transfer level (RTL) code, which is then synthesized into gate level hardware[7]. Dynamic scheduling without real time requirments focuses on improving performance metrics such as power consuption as we said before which is one of the most important requirments in todays era, apart from this it also focuses on resourse utilization and area[7]

B. Classification of Real Time Systems

Real times system are generally categorized into three main types: **Hard, firm and soft** Real Rime.

Hard real time system: which requires strict time constraints. Each operation must be completed within a spacific time frame to ensure system correctness and safty common domain like automotive control, medicle monitoring, aerospac or industrial automation. in these industries timing violation can lead to system failure, so schedulers are perticullarly designed to guarantee such deadlines[2, 8, 3].

firm real time system: by contrast, consider a task useless if the deadline is missed, but occasional deadline mmisses do not cause system big errors.[8, 3]

soft real time systems: such as vedio games, and in many non real time system such as porttable media player, consumer electrons and whireless sensor nodes, tasks do not need strict timing constrants. Insted what is important is that the overall throughput thatis the rate at which usefull work is completed over time. these systems allow greater flexibility in scheduling, which can usefull for improving other areas such as power consumption, area and hardware resuse[6, 8, 3].

C. Limitations of Traditional Scheduling

This report focuses on task scheduling techniques suitable for firm and soft real time systems, where meeting every deadline is not important but the quality should stay high and perform well. traditional real times system schceduling algorithms, such as Rate Monotonic (RMS) and Earliest deadline First (EDF), assume thatt all deadlines must be met and may not perform welll under overload condditions common in soft real time envirments[5]

D. Dynamic Scheduling Without Real Time Constraints

To address this, researches have introduced Dynamic scheduling without realtime requirments, that allow certain tasks to be skipped or delayed in order to keep the system stable under laod. One such approch is the Skip Over model, which allowes he system to skip executing spacific tasks. Another is Red as Late as Possible (RLP) algorithm, which combines deadlines deferral with selective execution to mmaximize the number of useful task completions while still preventing system overload[8]

In systems without hard tming constraints, dynamic schedluling witout real time requirments becomes a really powerful method fo managing tasks[7, 6]. Unlike systems that need operations to occure at exact clock cycles, dynamic schedulling gives the system more freedom[7]. the scheduler can decide when each task should run based on current conditions rather then following a fixed timeline[2]. that means tasks can be delayed or moved forward depending on resource availabilit or system load[7, 6]. as a result, the system can operate moree smoothly, avoid unecessary waiting, and make better use of available resources[7]. by not being locked on a rigid schedule, the system gains flexibility, which can lead to improvements in performance and efficincy without the need to meet deadlines[7].

E. PowerAware Scheduling Using Compatibility Graphs

Scheduling strategy introduced by Achim Rettberg in his dissertation Low Power Driven High-Level Synthesis for Dedicated Architectures[7]. in his works the statgey he represents where the system behaviour is broken into multiple execution paths, each representing a distinct flow op operations derived from control and data flow analysis. these paths are examined to determine mutual exclusivity that is sets of operations that are never active at the same time.

to find these mutally exclusive groups, Rettberg introduced the use of compatibility graph, where each node represents a path and edges indicate exclusivity. using clique detection algorithms, the scheduler indentifies sets of paths that can be grouped into power managed partions. Each partition is assigned a switch controller, which turns it on or off based on the current control flow conditions at runtime. Because only one of these partitions is active at a time, the system can significantly reduce dynamic and static power consumption without requiring any real-time timing guarantees[7]

II. Methodology

III. Constraints and Cost Function

IV.

V.

References

[1] Anantha P. Chandrakasan, Samuel Sheng, and Robert W. Brodersen. “Low-power CMOS digital design”. In: *IEEE Journal of Solid-State Circuits* 27.4 (1992). Supports motivation for low-power design in mobile/embedded systems. Cited in introduction to emphasize why power is equally important as speed in modern digital systems., pp. 473–484. DOI: 10.1109/4.126534.

[2] Chunhong Chen and Majid Sarrafzadeh. “Power-Manageable Scheduling Technique for Control-Dominated High-Level Synthesis”. In: *Design, Automation and Test in Europe (DATE)*. Used in discussion of control-heavy systems and introducing ”soft edge” slack. Shows how delaying operations slightly can create idle periods and enable power gating. Supports claim on resource-aware scheduling in flexible timing environments. IEEE, 2002, pp. 1016–1021.

[3] M. Hamdaoui and P. Ramanathan. “A Dynamic Priority Assignment Technique for Streams with (m,k)-firm Deadlines”. In: *IEEE Transactions on Computers* 44.12 (1995). Used to explain firm real-time model where m of k deadlines must be met. Helps justify dynamic task handling in systems with flexible deadline enforcement. Supports classification of real-time types in introduction., pp. 1443–1451. DOI: 10.1109/12.476316.

[4] Hubert Kaeslin. *Digital Integrated Circuit Design: From VLSI Architectures to CMOS Fabrication*. Provides foundational concepts on low-power VLSI and energy-efficient digital system design. Cambridge University Press, 2010.

[5] A. Koren and D. Shasha. “Skip-Over: Algorithms and Complexity for Overloaded Systems that Allow Skips”. In: *Proceedings of the 16th Real-Time Systems Symposium*. Introduces skip-over scheduling model. Used in explaining strategies for soft real-time systems where some deadlines can be missed without harming system behavior. Supports flexibility argument. IEEE, 1995.

[6] José C. Monteiro et al. “Scheduling Techniques to Enable Power Management”. In: *Proceedings of the 33rd Annual Design Automation Conference (DAC)*. Foundational work showing that scheduling flexibility can reduce power. Validates claim that delaying or advancing tasks helps power optimization. Supports discussion of early power-aware scheduling under no hard deadlines. ACM, 1996, pp. 349–352.

[7] Achim Rettberg. “Low Power Driven High-Level Synthesis for Dedicated Architectures”. Core source for this paper. Describes path-based partitioning for power-aware dynamic scheduling in HLS. Supports the method of compatibility graph construction, clique detection, and runtime switch control without real-time constraints. Used throughout the report including methodology and power optimization strategy. Dissertation. University of Paderborn, 2006.

[8] Y. Yoon and G. Manimaran. “RLP: Red as Late as Possible Strategy for Scheduling Skip-Over Real-Time Tasks in Dynamic Systems”. In: *2009 15th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*. Cited in section discussing soft/firm real-time systems and scheduling under overload. Supports idea of delaying/skipping tasks to maintain performance without deadline guarantees. Used to explain RLP model. Beijing, China: IEEE, 2009, pp. 291–298. DOI: 10.1109/RTCSA.2009.17.